

Quy hoạch động nén trạng thái dựa trên tính liên thông

Chen Danqi

Trường Trung học Yali, Trường Sa

Trại đông Tin học toàn quốc, 2008

Nguồn gốc: Dynamic programming with connectivity-based state compression

IOI China candidate team papers / 2008 / Day 2 / Chen Danqi

Tóm tắt

Quy hoạch động nén trạng thái là lớp bài toán dùng thông tin của một tập nhỏ làm trạng thái, nên số trạng thái thường tăng theo hàm mũ. Trên nền nén trạng thái ấy, có một lớp bài cần ghi thêm quan hệ liên thông giữa các phần tử. Bài viết gọi đó là quy hoạch động nén trạng thái dựa trên tính liên thông, và bàn về cách giải cũng như các hướng tối ưu của lớp bài này.

Nội dung được trình bày theo các bước quen thuộc của quy hoạch động: chia giai đoạn, xác định trạng thái, chuyển trạng thái và hiện thực chương trình. Qua các ví dụ *Formula 1*, *Formula 2*, *Black & White*, đếm cây khung và *Rocket Mania*, bài viết cũng nêu kinh nghiệm giảm số trạng thái và giảm chi phí chuyển.

Từ khóa: nén trạng thái; tính liên thông; biểu diễn ngoặc; đường biên; phích cắm; mô hình bàn cờ.

1 Dẫn nhập

Xét bài toán người bán hàng du lịch: cho đồ thị đầy đủ có trọng số gồm $n \leq 15$ đỉnh, cần tìm chu trình khép kín đi qua mỗi đỉnh đúng một lần với tổng trọng số nhỏ nhất. Đây là bài toán NP-khó, nhưng không có nghĩa là chỉ có tìm kiếm vét cạn. Tại mọi thời điểm, ta chỉ cần biết tập đỉnh đã đi qua và vị trí hiện tại; thứ tự cụ thể trước đó không ảnh hưởng tới quyết định sau. Vì vậy có thể đặt

$$f(i, S) = \min_{j \in S, j \neq i} \{f(j, S \setminus \{i\}) + w(j, i)\},$$

trong đó $i \in S$. Độ phức tạp $\mathcal{O}(n^2 2^n)$ vẫn là hàm mũ, nhưng với $n = 15$ tốt hơn rất nhiều so với tìm kiếm $\mathcal{O}(n!)$.

Những bài toán lấy thông tin của một tập nhỏ làm trạng thái như vậy thường được gọi là quy hoạch động nén trạng thái, hoặc quy hoạch động trên tập. Chúng có hai đặc điểm: một vài chiều dữ liệu rất nhỏ, và bài toán vẫn thỏa nguyên lý tối ưu cùng tính không hậu nghiệm.

Trong nén trạng thái thông thường, thông tin của các phần tử trong miền nhỏ thường độc lập. Nhưng có những bài chỉ biết một ô được chọn hay không là chưa đủ. Chẳng hạn, với ma trận $m \times n$, $m, n \leq 9$, mỗi ô có giá trị $a_{i,j}$, cần chọn một khối liên thông có tổng giá trị lớn nhất. Nếu duyệt ô từ trên xuống dưới, sau vài hàng đầu các ô đã chọn có thể tách thành nhiều khối, rồi ở các hàng sau mới nối lại thành một khối. Trạng thái chỉ lưu hàng trước gồm những ô được chọn sẽ không phân biệt được các tình huống này. Ta cần ghi thêm những ô nào đang thuộc cùng một thành phần liên thông.

Trong một đồ thị vô hướng, hai đỉnh liên thông nếu giữa chúng tồn tại một đường đi. Lớp bài trong bài viết này có quan hệ rất gần với mô hình đồ thị: Hamilton, cây khung, tô màu liên thông, đường đi đơn trên bàn cờ, v.v. Chúng hoặc trực tiếp nói về tính liên thông, hoặc giấu thông tin liên thông trong mô tả bài toán.

Các phần sau lần lượt trình bày:

- phương pháp chung qua ví dụ *Formula 1*;
- biểu diễn ngoặc cho đường đi và chu trình đơn;
- một lớp bài tô màu bàn cờ;
- cách tìm “đường biên” trong mô hình không phải bàn cờ;
- các kỹ thuật cắt tỉa cho bài toán tối ưu;
- tổng kết và ghi chú về nguồn.

2 Phương pháp chung

Quy hoạch động nén trạng thái liên thông thường có một khuôn khá cố định. Phần này dùng một bài kinh điển để mô tả khuôn ấy.

2.1 Ví dụ 1: *Formula 1*

Mô tả bài toán. Cho bàn cờ $m \times n$, một số ô là vật cản. Hỏi có bao nhiêu chu trình đi qua mỗi ô không phải vật cản đúng một lần. Giới hạn $m, n \leq 12$. Trong ví dụ gốc, bàn 4×4 có hai ô $(1, 1)$, $(1, 2)$ là vật cản và có 2 chu trình hợp lệ.

2.2 Chia giai đoạn

Đây là bài trên mô hình bàn cờ. Cấu trúc lưới làm bàn cờ trở thành “sân khấu” thường gặp nhất của quy hoạch động liên thông. Có ba cách chia giai đoạn:

- theo hàng: xét từng hàng từ trên xuống hoặc ngược lại;
- theo cột: xét từng cột từ trái sang phải hoặc ngược lại;
- theo ô: xét từng ô theo một thứ tự cố định, thường là từ trên xuống, từ trái sang phải.

Với *Formula 1*, chuyển theo hàng và theo cột là đối xứng. Bài gốc so sánh chuyển theo hàng với chuyển theo ô; các phần sau cũng mặc định dùng chuyển theo ô nếu không nói khác.

2.3 Phích cắm và đường biên

Một khái niệm then chốt là **phích cắm**. Với liên thông 4 hướng, mỗi ô có bốn phích cắm: trên, dưới, trái, phải. Phích cắm theo một hướng tồn tại nghĩa là ô đó nối ra ngoài theo hướng ấy. Vì ta cần một chu trình Hamilton trên các ô không bị chặn, mỗi ô hợp lệ phải có đúng hai trong bốn phích cắm.

Khi chuyển theo hàng, sau khi xử lý xong hàng i , thông tin ảnh hưởng tới hàng $i + 1$ gồm:

- với mỗi cột, hàng i có phích cắm xuống hay không;
- các phích cắm đó thuộc các thành phần liên thông nào.

Nếu chỉ ghi tồn tại phích cắm mà không ghi liên thông, thuật toán có thể sinh nhiều chu trình rời nhau, trong khi đề bài yêu cầu một chu trình duy nhất.

Đường phân cách giữa vùng đã quyết định và vùng chưa quyết định được gọi là **đường biên**. Chỉ các ô và phích cắm kề trực tiếp đường biên mới có thể ảnh hưởng tới phần chưa xử lý. Vì vậy trạng thái theo hàng có dạng

$$f(i, S_0, S_1),$$

trong đó S_0 là bitmask độ dài n cho biết phích cắm xuống của hàng i , còn S_1 mô tả liên thông giữa các vị trí trên đường biên.

Để biểu diễn liên thông, ta gán nhãn dương cho từng vị trí; hai vị trí cùng nhãn nếu thuộc cùng một thành phần. Nhãn 0 dành cho vị trí không có phích cắm hoặc ô bị chặn. Cùng một quan hệ liên thông có thể có nhiều dãy nhãn, ví dụ $\{1, 1, 2, 2\}$ và $\{2, 2, 1, 1\}$, nên cần chuẩn hóa. Một chuẩn hóa thông dụng là quét từ trái sang phải: thành phần chưa gặp đầu tiên mang nhãn 1, thành phần tiếp theo mang nhãn 2, v.v. Ví dụ các thành phần $(1, 2, 5)$, $(3, 6)$, (4) được mã hóa thành $\{1, 1, 2, 3, 1, 2\}$.

Có thể tối ưu thêm: nếu một ô trên đường biên không có phích cắm đi xuống thì liên thông của nó không còn tác động trực tiếp tới vùng chưa xử lý. Do đó thay vì lưu liên thông của ô, ta lưu liên thông của phích cắm. Phích cắm tồn tại thì mang nhãn thành phần; không tồn tại thì mang 0. Cách này làm trạng thái gọn hơn và giảm đáng kể số trạng thái hợp lệ.

Khi chuyển theo ô, sau khi xử lý xong ô (i, j) , đường biên cắt qua n ô và $n + 1$ phích cắm: n phích cắm xuống của các cột, cộng phích cắm phải của ô (i, j) . Trạng thái có thể viết là

$$f(i, j, S),$$

trong đó S mã hóa liên thông của $n + 1$ phích cắm trên đường biên.

2.4 Chuyển trạng thái

Chi phí chuyển gồm hai phần: số trạng thái kế tiếp của mỗi trạng thái hiện tại và thời gian tính mã trạng thái mới.

Với chuyển theo hàng, khi đi từ hàng i sang $i + 1$, ta phải liệt kê cách đặt phích cắm cho các ô của hàng mới. Mỗi ô không bị chặn có sáu mẫu phích cắm, nhưng nếu đã biết phích cắm trên và trái thì còn nhiều nhất hai lựa chọn. Sau khi liệt kê xong cả hàng, cần chuẩn hóa lại liên thông bằng DSU hoặc BFS/DFS trong $\mathcal{O}(n)$. Ngoài hàng cuối, mỗi thành phần liên thông đang mở phải có ít nhất một phích cắm xuống, nếu không nó sẽ bị tách khỏi phần còn lại vĩnh viễn.

Với chuyển theo ô, mỗi bước chỉ thay đổi hai vị trí trên đường biên: phích cắm phải của $(i, j - 1)$ trở thành phích cắm xuống của (i, j) , và phích cắm xuống của $(i - 1, j)$ trở thành phích cắm phải của (i, j) . Khi xét mẫu phích cắm của ô hiện tại, có ba kiểu chính:

1. **Tạo thành phần mới.** Ô có phích cắm phải và xuống. Hai phích cắm mới nối với nhau nhưng chưa nối với phích cắm khác. Cần gán nhãn mới và chuẩn hóa, thường tốn $\mathcal{O}(n)$.
2. **Hợp nhất hai thành phần.** Ô có phích cắm trên và trái. Nếu hai phích cắm thuộc hai thành phần khác nhau, hợp nhất chúng và chuẩn hóa. Nếu chúng đã cùng thành phần, ta vừa đóng một chu trình; điều này chỉ được phép tại ô không bị chặn cuối cùng.
3. **Kéo dài thành phần.** Trong hai phích cắm trên/trái có đúng một cái tồn tại, và trong hai phích cắm phải/xuống có đúng một cái tồn tại. Phích cắm mới kế thừa nhãn của phích cắm cũ, tốn $\mathcal{O}(1)$.

Khi đi từ cuối một hàng sang đầu hàng sau, đường biên cần xử lý riêng. So với chuyển theo hàng, chuyển theo ô có số nhánh kế tiếp hằng số và hằng số nhỏ hơn; đây là lý do nó thường hiệu quả hơn.

2.5 Hiện thực

Một trạng thái liên thông của $n + 1$ phích cắm có thể lưu bằng mảng, nhưng mảng khó bấm và tốn bộ nhớ. Thực tế thường mã hóa thành số trong hệ cơ số p , với p lớn hơn nhân thành phần lớn nhất có thể xuất hiện. Với *Formula 1*, có thể dùng cơ số 7, nhưng cơ số là lũy thừa của 2 thường nhanh hơn nhờ phép bit; bản gốc khuyên dùng cơ số 8.

Nếu cần đổi nhân trên diện rộng, giải mã trạng thái ra mảng, sửa mảng rồi mã hóa lại trong $\mathcal{O}(n)$. Nếu chỉ sửa vài vị trí, có thể lấy chữ số thứ i bằng phép chia lấy dư và thay trực tiếp chữ số ấy.

Có hai khung hiện thực thường gặp:

1. **Mã hóa toàn bộ trạng thái có thể có.** Sinh trước mọi trạng thái liên thông hợp lệ, đánh số $1, \dots, Tot$, rồi duyệt tất cả mã ở mỗi ô. Khung này rõ ràng nhưng có nhiều vòng lặp rỗng vì nhiều trạng thái không xuất hiện ở vị trí hiện tại.
2. **Mở rộng theo hàng đợi có nhớ.** Bắt đầu từ trạng thái ban đầu; mỗi lần lấy một trạng thái đang có, mở rộng các trạng thái kế tiếp và dùng bảng băm để gộp trạng thái trùng. Cách này chỉ duyệt trạng thái thật sự xuất hiện.

Hai khung có thể viết gọn như sau.

```
for i = 1..m:
  for j = 1..n:
    for k = 1..Tot:
      for x in transitions(i, j, State[k]):
        next[id(x)] += cur[k]
```

```
push all initial states into Queue
while Queue is not empty:
  p = pop Queue
  for x in transitions(p):
    if x was seen:
      sum[x] += sum[p]
    else:
      push x
      sum[x] = sum[p]
```

Bảng sau là số liệu thực nghiệm trong bản gốc, so sánh tổng trạng thái được mở rộng bởi hàng đợi với cách duyệt mã trực tiếp.

Dữ liệu	Hàng đợi	<i>Tot</i>	<i>Totmn</i>	Tỉ lệ vô hiệu
9×9 , (1, 1) là vật cản	30930	2188	177228	82.5%
10×10 , không vật cản	134011	5798	579800	76.8%
11×11 , (1, 1) là vật cản	333264	15511	1876831	82.2%
12×12 , không vật cản	1333113	41835	6024240	77.9%

Tỉ lệ trạng thái vô hiệu rất cao, đặc biệt khi bàn có vật cản, nên mở rộng bằng hàng đợi thường nhanh hơn. Một tối ưu bấm quan trọng là dùng bảng băm nhỏ và xóa sau mỗi ô (i, j) , chỉ bấm theo trạng thái x , thay vì dùng bảng băm lớn cho bộ (i, j, x) . Nếu không cần lưu đường đi, hàng đợi cũng có thể cuộn theo lớp.

Số liệu thời gian trong bản gốc:

Dữ liệu	P1	P2	P3	P4
10×10 , không vật cản	46ms	31ms	15ms	31ms
11×11 , (1, 1) là vật cản	140ms	499ms	109ms	187ms
12×12 , không vật cản	624ms	1840ms	499ms	873ms

Trong đó P1 là cơ số 8 và duyệt mã; P2 là cơ số 8, hàng đợi, bảng băm lớn; P3 là cơ số 8, hàng đợi, bảng băm nhỏ xóa theo ô; P4 giống P3 nhưng dùng cơ số 7. Kết quả cho thấy chi tiết hiện thực và hằng số có tác động rất lớn.

3 Đường đi đơn và biểu diễn ngoặc

Phần này xét các bài đường đi đơn và chu trình đơn trên bàn cờ. Đường đi đơn là đường đi không lặp đỉnh, trừ trường hợp đầu và cuối có thể trùng; chu trình đơn là đường đi đơn có đầu bằng cuối.

Với chu trình đơn, vùng đã xử lý phía trên đường biên gồm nhiều đường đi rời nhau không cắt nhau. Mỗi đường đi có đúng hai đầu mút, tương ứng với hai phích cắm trên đường biên. Vì các đường đi nằm trong mặt phẳng lưới, các cặp đầu mút không thể bắt chéo.

Tính chất. Giả sử trên đường biên từ trái sang phải có bốn phích cắm a, b, c, d . Nếu a liên thông với c , và b không liên thông với a , thì b không thể liên thông với d . Nếu ngược lại b nối với d , hai đường đi $a-c$ và $b-d$ trong vùng đã xử lý buộc phải cắt nhau, kéo theo bốn phích cắm cùng thuộc một thành phần, mâu thuẫn.

Tính chất “ghép đôi không bắt chéo” gọi tới ngoặc đúng. Ta đánh dấu phích cắm bên trái của mỗi cặp bằng ngoặc mở, bên phải bằng ngoặc đóng. Khi đó chỉ cần dùng ba ký hiệu:

$$0 = \text{không có phích cắm}, \quad 1 = \text{ngoặc mở}, \quad 2 = \text{ngoặc đóng}.$$

Nếu dùng dấu # cho vị trí rỗng, các trạng thái có dạng như $(())#()$, $(()\#)()$ hoặc $(()###)$. Đây là **biểu diễn ngoặc**.

3.1 Chuyển bằng biểu diễn ngoặc

Gọi p là phích cắm phải của $(i, j-1)$, q là phích cắm xuống của $(i-1, j)$, còn d và r là phích cắm xuống và phải mới của ô (i, j) . Ký hiệu $W(x) \in \{0, 1, 2\}$ là loại của phích cắm x .

Các trường hợp chính:

1. **Tạo đường đi mới.** $W(p) = W(q) = 0$, ô hiện tại dùng phích cắm phải và xuống. Đặt $W(d) = 1$, $W(r) = 2$. Chi phí $\mathcal{O}(1)$.
2. **Hợp nhất hai đường đi.** $W(p) > 0$, $W(q) > 0$, và cả hai phích cắm mới đều rỗng. Có bốn trường hợp:
 - $W(p) = 1, W(q) = 1$: tìm ngoặc đóng khớp với q và đổi nó thành ngoặc mở;
 - $W(p) = 2, W(q) = 2$: tìm ngoặc mở khớp với p và đổi nó thành ngoặc đóng;
 - $W(p) = 1, W(q) = 2$: nếu p, q khớp nhau, ta đóng một chu trình; chỉ hợp lệ ở ô không bị chặn cuối cùng;
 - $W(p) = 2, W(q) = 1$: hai cặp ngoài tự ghép đúng, không cần đổi vị trí khác.

Hai trường hợp đầu cần giải mã hoặc quét như mô phỏng ngăn xếp để tìm ngoặc khớp, tốn $\mathcal{O}(n)$; hai trường hợp sau tốn $\mathcal{O}(1)$.

3. **Kéo dài đường đi.** Trong p, q có đúng một phích cắm tồn tại, và trong d, r cũng đúng một phích cắm tồn tại. Phích cắm mới giữ nguyên loại ngoặc của phích cắm cũ, tồn $\mathcal{O}(1)$.

Biểu diễn ngoặc tận dụng tính chất mỗi thành phần có đúng hai phích cắm. Nó không làm giảm số trạng thái được mở rộng, nhưng giảm mạnh chi phí sửa nhân và làm hiện thực tự nhiên hơn. Thực tế thường dùng cơ số 4 thay cho cơ số 3 để thao tác bit nhanh hơn.

Dữ liệu	Min 7	Min 8	Ngoặc 3	Ngoặc 4
10×10 , không vật cản	31ms	15ms	0ms	0ms
11×11 , (1, 1) là vật cản	187ms	109ms	46ms	31ms
12×12 , không vật cản	873ms	499ms	265ms	140ms

Các bài tương tự gồm NWERC 2004 *Pipes*, HNOI 2004 *Postman*, HNOI 2007 *Park*. Một số bài đường đi không khép kín cũng có thể đổi về chu trình bằng cách sửa bàn cờ, chẳng hạn POJ 1739 *Tony's Tour*.

3.2 Ví dụ 2: *Formula 2*

Mô tả bài toán. Cho bàn cờ $m \times n$, một số ô là vật cản. Hỏi có bao nhiêu đường đi bắt đầu từ một ô không bị chặn, đi qua mọi ô không bị chặn đúng một lần. Giới hạn $m, n \leq 10$. Với bàn 2×2 không vật cản, có 4 đường đi hợp lệ.

Điểm khác với chu trình đơn là không phải mọi phích cắm đều ghép đôi với một phích cắm khác. Có những phích cắm nối với một đầu mút thật sự của đường đi; bản gốc gọi chúng là **phích cắm độc lập**. Ta mở rộng biểu diễn ngoặc thành cơ số 4:

$$0 = \text{rỗng}, \quad 1 = \text{ngoặc mở}, \quad 2 = \text{ngoặc đóng}, \quad 3 = \text{phích cắm độc lập}.$$

Các chuyển không liên quan tới phích cắm độc lập giống *Formula 1*. Các trường hợp mới:

1. Nếu $W(p) = W(q) = 0$, ô hiện tại có thể trở thành một đầu đường đi: đặt một trong d, r là 3, phích cắm còn lại rỗng.
2. Nếu $W(p) > 0, W(q) > 0$, cả d, r rỗng. Nếu p, q đều độc lập, nối chúng tạo thành một đường đi hoàn chỉnh; chỉ hợp lệ tại ô cuối cùng. Nếu chỉ một trong hai là độc lập, phích cắm khớp với phích cắm còn lại trở thành độc lập.
3. Nếu trong p, q chỉ có một phích cắm. Nếu nó độc lập, hoặc kéo dài bằng d hoặc r , hoặc tại ô cuối cùng biến nó thành đầu đường đi. Nếu nó là ngoặc mở/đóng, ta có thể “bịt” nó thành đầu đường đi, khi đó phích cắm khớp với nó đổi thành độc lập.

Mọi thời điểm có không quá hai phích cắm độc lập trên đường biên. Với 10×10 không vật cản, bản gốc ghi nhận khoảng 3493315 trạng thái được mở rộng, vẫn chấp nhận được.

3.3 Biểu diễn ngoặc tổng quát

Biểu diễn ngoặc thường yêu cầu mỗi thành phần có đúng hai phích cắm. Với bài tổng quát, một thành phần có thể có nhiều hơn hai phích cắm. Ta có thể dùng **biểu diễn ngoặc tổng quát**: trong một thành phần, phích cắm trái nhất được đánh dấu “(”, phích cắm phải nhất đánh dấu “)”, các phích cắm ở giữa đánh dấu “(” “)”. Nếu một thành phần chỉ có một phích cắm, đánh dấu bằng một cặp “()” tại chính vị trí đó.

Ví dụ trạng thái liên thông

$$\{1, 2, 2, 3, 4, 3, 2, 1\}$$

có thể diễn tả bằng chuỗi ngoặc tổng quát

$$(-(-)(-(-())-)-).$$

Cách này thường cần cơ số 4 hoặc 5. Vì các trường hợp sửa trực tiếp khá nhiều, khi cài đặt nên giải mã trạng thái ra mảng rồi mã hóa lại sau khi chuyển.

4 Một lớp bài tô màu bàn cờ

Xét các bài cho bàn cờ $m \times n$, cần tô mỗi ô bằng một trong k màu sao cho mọi ô cùng màu tạo thành một vùng liên thông 4 hướng. Phần này dùng *Black & White* làm ví dụ.

4.1 Ví dụ 3: *Black & White*

Mô tả bài toán. Một số ô của bàn $m \times n$ đã được tô đen hoặc trắng. Cần tô các ô còn lại đen hoặc trắng sao cho:

- tất cả ô đen liên thông, tất cả ô trắng liên thông;
- không có hình vuông con 2×2 mà cả bốn ô cùng màu.

Hỏi số cách tô, với $m, n \leq 8$. Ví dụ gốc có bàn 2×3 với một số ô chưa tô và có 9 phương án.

4.2 Trạng thái và chuyển

Với đường đi, hai ô kề nhau có liên thông hay không phụ thuộc vào phích cắm. Với tô màu, hai ô kề nhau liên thông nếu chúng cùng màu. Vì vậy trạng thái cần ghi màu của n ô trên đường biên và quan hệ liên thông của chúng.

Sau khi xử lý ô (i, j) , thông tin ảnh hưởng tới tương lai gồm:

- màu của n ô ngay trên đường biên;
- liên thông giữa các ô ấy;
- thêm màu của ô $(i - 1, j + 1)$, vì ràng buộc không có khối 2×2 đồng màu ảnh hưởng tới màu của ô kế tiếp.

Có thể viết trạng thái là

$$f(i, j, S_0, S_1, c_p),$$

trong đó S_0 là bitmask màu, S_1 là trạng thái liên thông, và c_p là màu phụ cần nhớ.

Bản gốc cũng nêu một tình huống: nếu hai ô kề nhau không cùng thành phần thì chúng chắc chắn khác màu, nên trong một số tình huống có thể suy ra phần lớn màu từ S_1 và chỉ cần ghi màu của hai ô biên. Tuy nhiên tình huống này không giảm số trạng thái đáng kể, nên chỉ có ý nghĩa tham khảo.

Khi chuyển, liệt kê màu của ô (i, j) . Bitmask màu và màu phụ cập nhật trong $\mathcal{O}(1)$. Phần liên thông S_1 được cập nhật bằng cách thay ô $(i - 1, j)$ trên đường biên bằng ô (i, j) , rồi xét quan hệ màu của ô hiện tại với ô trên và ô trái:

```
decode S1 into c[1..n]
```

```
if (i,j) differs from (i-1,j) and (i-1,j) is a singleton:
```

```
    handle the closed component specially
```

```
else if (i,j) has same color as both upper and left cells:
```

```

merge their components
else if (i,j) differs from both upper and left cells:
    create a new component at c[j]
else if (i,j) has same color as left but differs from upper:
    c[j] = c[j-1]
renormalize c[1..n] and encode the new S1

```

Trường hợp đặc biệt quan trọng là ô $(i-1, j)$ bị rời khỏi đường biên trong khi nó là một thành phần đơn lẻ và khác màu với ô hiện tại. Khi đó thành phần này không thể nối thêm về sau; mọi ô còn lại của màu ấy sẽ làm vi phạm liên thông, nên cần kiểm tra riêng.

Với bàn 8×8 chưa tô, bản gốc ghi nhận 122395 trạng thái được mở rộng; mỗi chuyển tốn $\mathcal{O}(n)$, nên tổng chi phí khoảng 979160, chạy dưới 0.1 giây trong môi trường thử nghiệm. Các bài tương tự gồm tuyển chọn Trùng Khánh 2007 *Rect* và IPSC 2007 *Delicious Cake*.

4.3 Mở rộng và trạng thái vô hiệu

Nếu yêu cầu liên thông 8 hướng thay vì 4 hướng, hai ô chỉ cần chung một đỉnh. Khi đó phải ghi thông tin của mọi ô có ít nhất một đỉnh nằm trên đường biên, tức $n+1$ ô, bao gồm cả $(i-1, j)$.

Số trạng thái vô hiệu ảnh hưởng lớn tới hiệu suất. Với bài tô k màu, nếu từ trái sang phải có bốn thành phần không lồng nhau a, b, c, d , trong đó a, c cùng màu, b, d cùng màu khác, thì trạng thái đó không thể hoàn tất thành lời giải liên thông. Khái niệm “lồng nhau” có thể hiểu bằng biểu diễn ngoặc tổng quát. Những kiểm tra như vậy giúp cắt trạng thái sớm.

5 Mô hình không phải bàn cờ

Không phải bài liên thông nào cũng nằm trên lưới, nhưng ý tưởng đường biên vẫn có thể dùng được nếu tìm được một thứ tự xử lý tốt.

5.1 Ví dụ 4: Đếm cây khung

Mô tả bài toán. Cho đồ thị vô hướng liên thông n đỉnh. Với hai đỉnh khác nhau i, j , có cạnh $\langle i, j \rangle$ nếu và chỉ nếu $|i - j| \leq k$. Biết n, k , cần đếm số cây khung. Giới hạn $n \leq 10^{15}$, $2 \leq k \leq 5$.

Phân tích. Điểm nổi bật là n rất lớn nhưng k rất nhỏ. Một cây khung cần hai tính chất: không có chu trình và liên thông. Nếu xét các đỉnh theo thứ tự $1, 2, \dots, n$, khi thêm đỉnh mới ta chọn nó nối với những đỉnh trước nào, đồng thời tránh tạo chu trình. Cuối cùng mọi đỉnh phải nằm trong một thành phần.

Sau khi đã quyết định các cạnh liên quan tới $1, \dots, i$, các đỉnh $1, \dots, i-k$ sẽ không còn cạnh tới đỉnh lớn hơn i . Vì vậy tương lai chỉ cần biết liên thông của k đỉnh

$$i-k+1, \dots, i.$$

Đây chính là đường biên của bài toán. Đặt $f(i, S)$ là số cách sau khi xét xong i đỉnh, trong đó S là liên thông của k đỉnh cuối.

Khi thêm đỉnh i , liệt kê 2^k tập cạnh nối từ i tới $i-1, \dots, i-k$. Để tránh chu trình, trong mỗi thành phần cũ, i được nối với nhiều nhất một đỉnh. Để tránh một thành phần bị bỏ lại vĩnh viễn, nếu $i-k$ là thành phần đơn lẻ trên đường biên thì i phải nối với $i-k$. Mỗi chuyển xử lý được trong $\mathcal{O}(2^k k)$.

Gọi T_k là số trạng thái liên thông khác bản chất của k điểm. Tìm kiếm cho $T_5 = 52$. Quy hoạch động trực tiếp có độ phức tạp $\mathcal{O}(n T_k 2^k k)$, vẫn quá lớn vì n có thể tới 10^{15} . Nhưng khi $i \geq k$, chuyển từ trạng

thái này sang trạng thái kia chỉ phụ thuộc vào trạng thái, không phụ thuộc i . Do đó có thể dựng ma trận chuyển và dùng lũy thừa ma trận. Độ phức tạp còn $\mathcal{O}(T_k^3 \log n)$, đủ với $k = 5$.

Điểm tổng quát rút ra là: nếu có thể sắp thứ tự các đỉnh sao cho mọi cạnh nối hai đỉnh cách nhau không quá p , với p nhỏ, thì sau khi xử lý tiền tố chỉ cần ghi liên thông của p đỉnh cuối. Với bàn cờ, thứ tự từ trái sang phải, từ trên xuống dưới có p bằng số cột hoặc số hàng; vì vậy ít nhất một chiều của bàn phải nhỏ.

6 Cắt tỉa trong bài toán tối ưu

Hiệu suất của quy hoạch động liên thông phụ thuộc chủ yếu vào số trạng thái và chi phí chuyển. Phần này dùng *Rocket Mania* để minh họa cách cắt tỉa trạng thái trong bài tối ưu.

6.1 Ví dụ 5: *Rocket Mania*

Mô tả bài toán. Bài toán lấy bối cảnh trò chơi giả tưởng “pháo hoa Trung Hoa”. Có bàn 9×6 . Bên trái có 9 que diêm, bên phải có 9 tên lửa. Mỗi ô có thể rỗng hoặc chứa một đoạn ống thuộc một trong bốn loại: chữ L, đoạn thẳng, chữ T, hoặc dấu cộng. Một tên lửa bắn được nếu tồn tại đường ống nối từ một que diêm đang cháy tới tên lửa đó.

Cho trạng thái ban đầu và chỉ số X . Ta được xoay mỗi đoạn ống 0, 90, 180, hoặc 270 độ. Mục tiêu là khi đốt que diêm thứ X , số tên lửa bắn được là lớn nhất.

6.2 Trạng thái cơ bản

Duyệt từng ô từ trái sang phải, từ trên xuống dưới. Cần ghi:

- liên thông của 10 phích cắm trên đường biên, trong đó trạng thái liên thông cũng cho biết phích cắm có tồn tại hay không;
- bitmask *fired* độ dài 10, cho biết phích cắm nào đã được đốt.

Trạng thái có thể viết là

$$f(i, j, S, \text{fired}) \in \{\text{đạt được}, \text{không đạt được}\}.$$

Đáp án cuối là giá trị lớn nhất của $\text{Ones}(\text{fired})$ trên mọi trạng thái đạt được, trong đó Ones đếm số bit 1.

Mỗi ô liệt kê tối đa bốn góc xoay, rồi xét có nối với ô trên và ô trái hay không. Cách chuyển giống các bài bàn cờ trước, thường cần $\mathcal{O}(m)$ để sửa liên thông. Nếu làm trực tiếp, ngay cả dữ liệu mẫu trong bản gốc cũng chạy hơn 60 giây, nên cắt tỉa là bắt buộc.

6.3 Bốn kiểu cắt tỉa

Cắt tỉa có hai loại: cắt theo tính khả thi, tức trạng thái không thể dẫn tới lời giải hợp lệ; và cắt theo tính tối ưu, tức trạng thái không thể tốt hơn một trạng thái khác.

1. Nếu mọi phích cắm trên đường biên đều chưa được đốt, về sau không tên lửa nào có thể bắn được; bỏ trạng thái này. Cắt tỉa này rất đơn giản nhưng trên nhiều dữ liệu bỏ được gần một nửa trạng thái.
2. Nếu có phích cắm p chưa được đốt và không liên thông với phích cắm nào khác, coi p là vô hiệu và xóa nó. Ngay cả khi đường chứa p sau này được đốt và dẫn tới tên lửa, sẽ tồn tại đường khác không cần đi qua p mà vẫn đạt cùng hiệu quả. Đây là cắt tỉa quan trọng nhất; nhiều dữ liệu giảm số trạng thái từ bảy, tám lần tới hơn mười lần.

3. Với cùng ô (i, j) , nếu trạng thái thứ hai có mọi phích cắm tồn tại của trạng thái thứ nhất, và các phích cắm ấy trong trạng thái thứ hai đều đã được đốt, thì trạng thái thứ hai không kém trạng thái thứ nhất trong mọi tương lai. Ta có thể bỏ trạng thái thứ nhất. Bản gốc dùng một trạng thái *Best* có Ones(*fired*) lớn nhất để lọc nhanh các trạng thái chắc chắn không tốt hơn.
4. Biên phải của bàn cờ tạo ra nhiều trạng thái vô hiệu. Sau khi xử lý ô ở cột cuối, nếu một phích cắm thuộc nhóm biên không được đốt và không có phích cắm nhóm còn lại liên thông với nó, thì phích cắm ấy cũng vô hiệu.

Bản gốc có bảng thực nghiệm cho 10 bộ dữ liệu, lần lượt thêm các chốt tĩa 1, 2, 3, 4. Tập Word cũ làm mất ranh giới cột của bảng này khi trích văn bản, nên bản dịch không chép lại các số rời rạc để tránh sai lệch. Kết luận của bảng rất rõ: sau khi thêm bốn chốt tĩa, số trạng thái giảm mạnh; những chốt tĩa có vẻ nhỏ đôi khi tạo khác biệt rất lớn. Sau tối ưu, hơn 60 bộ kiểm thử chạy dưới 1.5 giây trong môi trường của tác giả.

Về biểu diễn S , nếu dùng chuẩn hóa nhân thông thường cần cơ số 10. Nhờ chốt tĩa thứ hai, một phích cắm là thành phần đơn lẻ thì nó chắc chắn đã được đốt. Khi dùng biểu diễn ngoặc tổng quát, có thể để trạng thái rỗng và trạng thái thành phần đơn lẻ cùng dùng dạng “()”, rồi dựa vào *fired* để phân biệt. Nhờ đó chỉ cần bốn ký hiệu chính: “(”, “)”, “(”, “)”, có lợi về hằng số so với cơ số 10.

Bài còn có bản mở rộng *Rocket Mania Plus*: khác biệt duy nhất là tất cả que diêm bên trái đều được đốt. Khi đó chỉ cần đặt ban đầu 9 phích cắm phải tương ứng là đã đốt và nằm trong một thành phần.

7 Tổng kết

Bài viết xoay quanh hai việc: cách giải và cách tối ưu quy hoạch động nén trạng thái có liên thông.

Phương pháp chung là tìm đường biên, chỉ giữ thông tin trên đường biên có thể ảnh hưởng tới tương lai, rồi chuẩn hóa quan hệ liên thông. Với bài đường đi và chu trình đơn, tính chất không bắt chéo cho phép thay nhân liên thông bằng biểu diễn ngoặc, giảm mạnh hằng số chuyển trạng thái. Khi thành phần có nhiều phích cắm, biểu diễn ngoặc tổng quát giữ lại cùng tinh thần nhưng cần hiện thực cẩn thận hơn. Với bài tô màu, trạng thái phải bổ sung màu; với mô hình không phải bàn cờ, điều quan trọng là tìm một thứ tự đỉnh sao cho đường biên nhỏ. Với bài tối ưu, chốt tĩa theo khả thi và ưu thế trạng thái thường quyết định thuật toán có chạy được hay không.

Chi tiết cài đặt không phải phụ. Chọn cơ số mã hóa, thời điểm giải mã, cách băm, cách cuộn hàng đợi và các chốt tĩa nhỏ đều có thể làm thay đổi hiệu suất theo bội số lớn. Vì vậy khi dùng kỹ thuật này, cần vừa nắm mô hình tổng quát, vừa phân tích kỹ đặc trưng riêng của từng bài.

Tài liệu tham khảo

- Liu Rujia, Huang Liang, *The Art of Algorithms and Informatics Olympiad*.
- Jin Kai, bài tập đội tuyển quốc gia năm 2004, báo cáo lời giải *Black & White*.
- Mao Ziqing, luận văn đội tuyển quốc gia năm 2001, *Optimization techniques for dynamic programming algorithms*.
- UVA Online Judge: <http://icpcres.ecs.baylor.edu/onlinejudge>
- Ural Online Judge: <http://acm.timus.ru>
- ZJU Online Judge: <http://acm.zju.edu.cn>

Lời cảm ơn

Tác giả cảm ơn thầy Zhu Quanmin vì đã hướng dẫn trong quá trình viết bài, cảm ơn huấn luyện viên Liu Rujia vì những chỉ dẫn và gợi mở, cảm ơn Liu Chenheng và Jin Bin vì đã giúp ích nhiều cho bản thảo, cũng như các thành viên đội tuyển Zheng Dun, Zhou Dong, Yu Linyun, Yu Huacheng, Gu Yan, Zhou Mengyu và Xiao Hanjun.

Ghi chú về nguồn

Tệp .doc gốc có nhiều hình minh họa, công thức nhúng và phụ lục đề tiếng Anh. Bản dịch giữ nội dung chính, diễn giải hình bằng lời, phục dựng các công thức chuẩn có thể suy ra từ ngữ cảnh, giữ các bảng số còn đọc được, và không chép dài các phụ lục đề bài.